VŠB TECHNICKÁ UNIVERZITA OSTRAVA | FAKULTA ELEKTROTECHNIKY A INFORMATIKY

Absolvování individuální odborné praxe

Individual Professional Practice in the Company

Michal Křižák

Bakalářská práce

Vedoucí práce: Ing. Gaura Jan, Ph.D.

Ostrava, 2026

Č.j.VSB/25/137510
vsboee2504056B

Zadání bakalářské práce

Student: **Michal Křižák**

Studijní program: B0613A140014 Informatika

Téma: Absolvování individuální odborné praxe
Individual Professional Practice in the Company

Jazyk vypracování: čeština

Zásady pro vypracování:

Student vykoná individuální praxi ve firmě: AVE Soft s.r.o.

Struktura závěrečné zprávy:
1. Popis odborného zaměření firmy, u které student vykonal odbornou praxi a popis pracovního zařazení studenta.
2. Seznam úkolů zadaných studentovi v průběhu odborné praxe s vyjádřením jejich časové náročnosti.
3. Zvolený postup řešení zadaných úkolů.
4. Teoretické a praktické znalosti a dovednosti získané v průběhu studia uplatněné studentem v průběhu odborné praxe.
5. Znalosti či dovednosti scházející studentovi v průběhu odborné praxe.
6. Dosažené výsledky v průběhu odborné praxe a její celkové zhodnocení.

Seznam doporučené odborné literatury:

Podle pokynů konzultanta, který vede odbornou praxi studenta.

Formální náležitosti a rozsah bakalářské práce stanoví pokyny pro vypracování zveřejněné na webových stránkách fakulty.

Vedoucí bakalářské práce: **Ing. Jan Gaura, Ph.D.**

Konzultant bakalářské práce: Ing. Martin Prokeš

Datum zadání: 01.09.2025

Datum odevzdání: 30.04.2026

Garant studijního programu: Ing. Tomáš Fabián, Ph.D.

V IS EDISON zadáno: 01.10.2025 16:58:20

Abstrakt

Tato zpráva shrnuje průběh individuální odborné praxe vykonané ve společnosti AveSoft. Hlavní náplní praxe byl vývoj a modernizace modulu Evolio – Importér, který slouží pro hromadný import dat do informačního systému. Cílem bylo nahradit stávající desktopové řešení (WPF) novou webovou Single Page Aplikací (SPA).

Práce se zaměřuje na implementaci uživatelského rozhraní v technologii Angular, které umožňuje uživatelům definovat šablony importů a mapovat sloupce z Excel souborů. Dále popisuje vývoj backendové logiky ve formě C# doplňku (Add-in) pro samotné zpracování dat a spolupráci na návrhu databázové struktury. Součástí praxe byla také tvorba automatizovaných testů a řešení dalších úloh v rámci firemního vývojového cyklu.

Klíčová slova

Odborná praxe; AveSoft; IS Evolio; Angular; TypeScript; C#; SQL; automatizované testy; migrace software; webová aplikace

Abstract

This report summarizes the individual professional practice performed at AveSoft company. The main focus of the practice was the development and modernization of the "Evolio – Importer" module, designed for bulk data import into the information system. The goal was to replace the existing desktop solution (WPF) with a new web-based Single Page Application (SPA).

The thesis focuses on the implementation of the user interface using Angular technology, enabling users to define import templates and map columns from Excel files. It further describes the development of backend logic in the form of a C# Add-in for data processing and collaboration on the database structure design. The practice also included creating automated tests and solving other tasks within the company's development cycle.

Keywords

Professional practice; AveSoft; IS Evolio; Angular; TypeScript; C#; SQL; automated tests; software migration; web application

Poděkování

Rád bych na tomto místě poděkoval vedoucímu práce, panu Ing. Janu Gaurovi, Ph.D., za odborné vedení a cenné rady. Velké poděkování patří také společnosti AveSoft s.r.o. za možnost absolvovat odbornou praxi, a především mým kolegům z vývojového týmu za vstřícný přístup, trpělivost a odbornou pomoc při řešení zadaných úkolů.

Obsah

Seznam použitých symbolů a zkratek **7**

Seznam obrázků **8**

Seznam tabulek **9**

1 Představení společnosti a pracovní pozice **11**
1.1 Profil společnosti AVE Soft s.r.o. 11
1.2 Pracovní zařazení a role v týmu 11

2 Zadání a harmonogram praxe **13**
2.1 Specifikace zadaných úkolů 13
2.2 Časový plán a náročnost 14

3 Realizace projektu Importér **15**
3.1 Analýza a návrh databáze 15
3.2 Serverová část a aplikační logika (Add-in) 17
3.3 Uživatelské rozhraní (Frontend) 20

4 Vývoj AI rozhraní a řešení FE issues **27**
4.1 Implementace AI asistenta v prostředí AngularJS 27
4.2 Řešení chyb v uživatelském rozhraní (Bugfixing) 31

5 Automatizované testování **33**
5.1 Strategie a nástroje 33
5.2 Implementace testovacích scénářů 34
5.3 Integrace do CI/CD (Azure DevOps) 34

6 Vazba praxe na studium **36**
6.1 Uplatnění teoretických znalostí 36
6.2 Přechod od teorie k praxi 37

7 Rozvoj kompetencí a chybějící znalosti **38**
7.1 Nově osvojené technologie 38
7.2 Chybějící procesní znalosti 39

8 Zhodnocení výsledků a přínos praxe **40**
8.1 Modul Importér 40
8.2 AI Rozhraní 40
8.3 Automatizované testy 40

9 Závěr **41**

Literatura **42**

Přílohy **42**

Seznam použitých zkratek a symbolů

IS	–	Informační systém
WPF	–	Windows Presentation Foundation
SPA	–	Single Page Application
FE	–	Frontend (prezentační vrstva aplikace)
BE	–	Backend (logická/datová vrstva aplikace)
API	–	Application Programming Interface
REST	–	Representational State Transfer
SQL	–	Structured Query Language
UI	–	User Interface (uživatelské rozhraní)
HTML	–	HyperText Markup Language
CSS	–	Cascading Style Sheets
GIT	–	Distribuovaný systém správy verzí
AI	–	Artificial Intelligence (Umělá inteligence)
LLM	–	Large Language Model (Velký jazykový model)
UX	–	User Experience (Uživatelský prožitek)
CI/CD	–	Continuous Integration / Continuous Deployment (Průběžná integrace a nasazování)
JSON	–	JavaScript Object Notation
HTTP	–	Hypertext Transfer Protocol
CRUD	–	Create, Read, Update, Delete (Základní databázové operace)
RxJS	–	Reactive Extensions for JavaScript
POM	–	Page Object Model
ERD	–	Entity-Relationship Diagram

Seznam obrázků

3.1 Zjednodušený ER diagram databázového schématu Importer 16
3.2 Ukázka rozhraní původního desktopového modulu (neintuitivní a složité). V rozbaleném dropdownu lze vidět entity, které jsou pro běžného uživatele nicneříkající. . . 20
3.3 Dashboard nového modulu Importér se seznamem náhledem dokumentu a výběrem šablony 21
3.4 Sekvenční diagram procesu zpracování importu po jednotlivých řádcích 22
3.5 Nové rozhraní pro interaktivní konfiguraci mapování sloupců 25
3.6 Ukázka úspěšného importu 26

4.1 Výchozí rozhraní AI asistenta s nabídkou předdefinovaných šablon 28
4.2 Ukázka probíhající konverzace s formátováním textu a analýzou dokumentu 30
4.3 Ukázka zadaného issue na gitlabu 31

5.1 Ukázka úspěšného běhu automatizovaných testů v release pipeline Azure DevOps . . 35

Seznam tabulek

3.1 Srovnání fází PrepareImport a DirectImport 18
3.2 Kompletní přehled komponent modulu Importér 21

Seznam použitých kódů

3.1 Implementace metody pro provedení ostrého importu 18
3.2 Uložená procedura pro aktualizaci stavu položek 19
3.3 Využití signálů pro správu stavu modulu 23
3.4 Paralelní inicializace dat pomocí forkJoin 23
3.5 Definice komponenty s OnPush strategií 23
4.1 Zpracování streamované odpovědi v AngularJS 29
4.2 Oprava konfliktu vlastností Fixed a Grouped 32
5.1 Ukázka testu vytvoření subjektu pomocí Screenplay patternu 34

Kapitola 1

Představení společnosti a pracovní pozice

Tato kapitola představuje společnost AVE Soft s.r.o., ve které probíhala odborná praxe, a popisuje pracovní zařazení v rámci vývojového týmu.

1.1 Profil společnosti AVE Soft s.r.o.

Společnost AVE Soft s.r.o. je česká softwarová firma se sídlem v Ostravě, která působí na trhu informačních technologií již od roku 1997. Firma se specializuje na vývoj komplexních informačních systémů, přičemž její vlajkovou lodí je systém **Evolio**, určený pro advokátní kanceláře a správu pohledávek. Kromě toho se společnost věnuje vývoji zakázkového softwaru, mobilních aplikací a vlastních inovativních produktů (např. Taxlio pro daňové poradce).

Sídlo společnosti se nachází v technologickém areálu v Ostravě-Pustkovci, což umožňuje úzkou spolupráci s akademickou sférou, zejména s VŠB – Technickou univerzitou Ostrava. Firma klade důraz na využití moderních technologií, jako jsou cloudové služby Microsoft Azure, platforma .NET [1] a frameworky pro vývoj webových aplikací (Angular).

1.2 Pracovní zařazení a role v týmu

V rámci odborné praxe jsem byl zařazen do vývojového týmu jako programátor. Moje práce probíhala pod vedením zkušeného konzultanta (mentora) z firmy. Hlavním cílem mého působení byla práce na projektu **Importér**, který je součástí širšího ekosystému Evolio.

Náplň práce

Moje činnost byla rozdělena do několika klíčových oblastí:

- **Vývoj frontendu (FE):** Práce na uživatelském rozhraní v technologii **Angular** [2] a jazyce **TypeScript** [3]. Hlavním úkolem bylo vytvoření moderní Single Page Application (SPA), která nahrazuje starší desktopové řešení.
- **Vývoj backendu (BE):** Implementace doplňku (Add-in) pro Importér v jazyce **C#** [4], který zajišťuje logiku zpracování dat a komunikaci s databází.
- **Návrh databáze:** Spolupráce na analýze a návrhu struktur nových databázových tabulek potřebných pro fungování importního modulu.
- **Automatizované testování:** Tvorba automatizovaných testů v jazyce C# pro zajištění kvality a stability vyvíjeného řešení.
- **Řešení FE issues:** Údržba a rozvoj existujících částí aplikace mimo hlavní projekt. Významným úkolem v této oblasti byla implementace **AI rozhraní** s chatem, která zahrnovala výběr jazykových modelů a práci s historií konverzace.

Jako člen týmu jsem se účastnil pravidelných porad a využíval firemní nástroje pro správu úkolů a verzování kódu.

Kapitola 2

Zadání a harmonogram praxe

Tato kapitola detailně specifikuje zadané úkoly, které byly předmětem odborné praxe, a popisuje časový harmonogram jejich plnění. Práce byla rozdělena do tří hlavních oblastí: vývoj nového modulu Importér, údržba a rozvoj stávajícího frontendu (včetně implementace AI) a tvorba automatizovaných testů.

2.1 Specifikace zadaných úkolů

2.1.1 Projekt Importér

Hlavním úkolem praxe byl vývoj webového rozhraní pro import dat do systému Evolio. Původní řešení, realizované jako desktopová aplikace (WPF), se v průběhu času ukázalo jako uživatelsky nepřívětivé a složité na pochopení. Jeho obsluhu zvládali pouze zaškolení firemní konzultanti, což bránilo běžným uživatelům v samostatné práci.

Cílem nového řešení (Single Page Application v Angularu) bylo vytvořit intuitivní nástroj, který umožní každému uživateli bez problémů importovat data samostatně. Jednalo se o vývoj zcela nové komponenty bez návaznosti na existující frontendový kód.

Klíčové požadavky:

- **Intuitivní UX/UI:** Důraz na jednoduchost, aby uživatelé mohli sami importovat entity jako jsou advokátní a realitní spisy, pohledávky, subjekty, platby a finanční položky.
- **Implementace designu:** Kódování aplikace podle dodaných grafických návrhů s možností vlastních vylepšení (např. implementace skeleton loadingů pro lepší odezvu aplikace).
- **Backend a Databáze:** Spolupráce na návrhu databázových struktur a vývoj C# doplňku (Add-in) pro zpracování dat.

2.1.2 Vývoj AI rozhraní a řešení FE issues

Druhou oblastí byla práce na stávajícím frontendu systému Evolio. Nejvýznamnějším úkolem v této části byla implementace uživatelského rozhraní pro AI asistenta.

Specifika AI modulu:

- Vývoj frontendu pro chatovací okno s podporou výběru jazykového modelu.
- Komunikace s backendem prostřednictvím REST API a **Notification Hubu** pro real-time streamování odpovědí od AI.
- Implementace funkce pro nahrávání a analýzu dokumentů, což byla klíčová funkce vyžadovaná klienty pro právní analýzy.

Kromě toho jsem se podílel na řešení běžných chyb (bugfixů) a drobných úprav v systému dle požadavků klientů.

2.1.3 Automatizované testování

Třetím pilířem praxe byla tvorba automatizovaných testů pomocí frameworku **Selenium**. Cílem bylo zajistit stabilitu klíčových částí systému Evolio. Testy pokrývají základní CRUD operace (Create, Read, Update, Delete) například pro moduly:

- Podatelna,
- Fakturace,
- Správa případů a spisů,
- Evidence subjektů a plateb.

2.2 Časový plán a náročnost

Odborná praxe byla zahájena v srpnu 2025 a probíhá kontinuálně až do současnosti. Práce na jednotlivých úkolech neprobíhala striktně lineárně, ale prolínala se podle aktuálních priorit vývojového týmu.

Přibližné rozložení časové dotace:

- **60 %** – Vývoj modulu Importér (FE + Add-in).
- **20 %** – Implementace AI rozhraní a řešení FE issues.
- **20 %** – Tvorba automatizovaných testů a ostatní činnosti.

Vývoj Importéru začal implementací frontendu. Práce na backendové části (C# Add-in) byla zahájena později, jakmile byla klientská část dostatečně připravena pro integraci. Následně probíhal vývoj obou částí souběžně.

Kapitola 3

Realizace projektu Importér

Tato kapitola popisuje technickou realizaci klíčové části odborné praxe – modulu Importér. Řešení je rozděleno do tří vrstev: databázové, aplikační (backend) a prezentační (frontend).

3.1 Analýza a návrh databáze

Databázová vrstva je implementována na platformě Microsoft SQL Server [5] pod vyhrazeným schématem `importer`. Architektura byla navržena s důrazem na konfigurovatelnost, což umožňuje definovat nové typy importů bez nutnosti zásahů do zkompilovaného kódu aplikace.

Model dat se skládá ze tří logických celků:

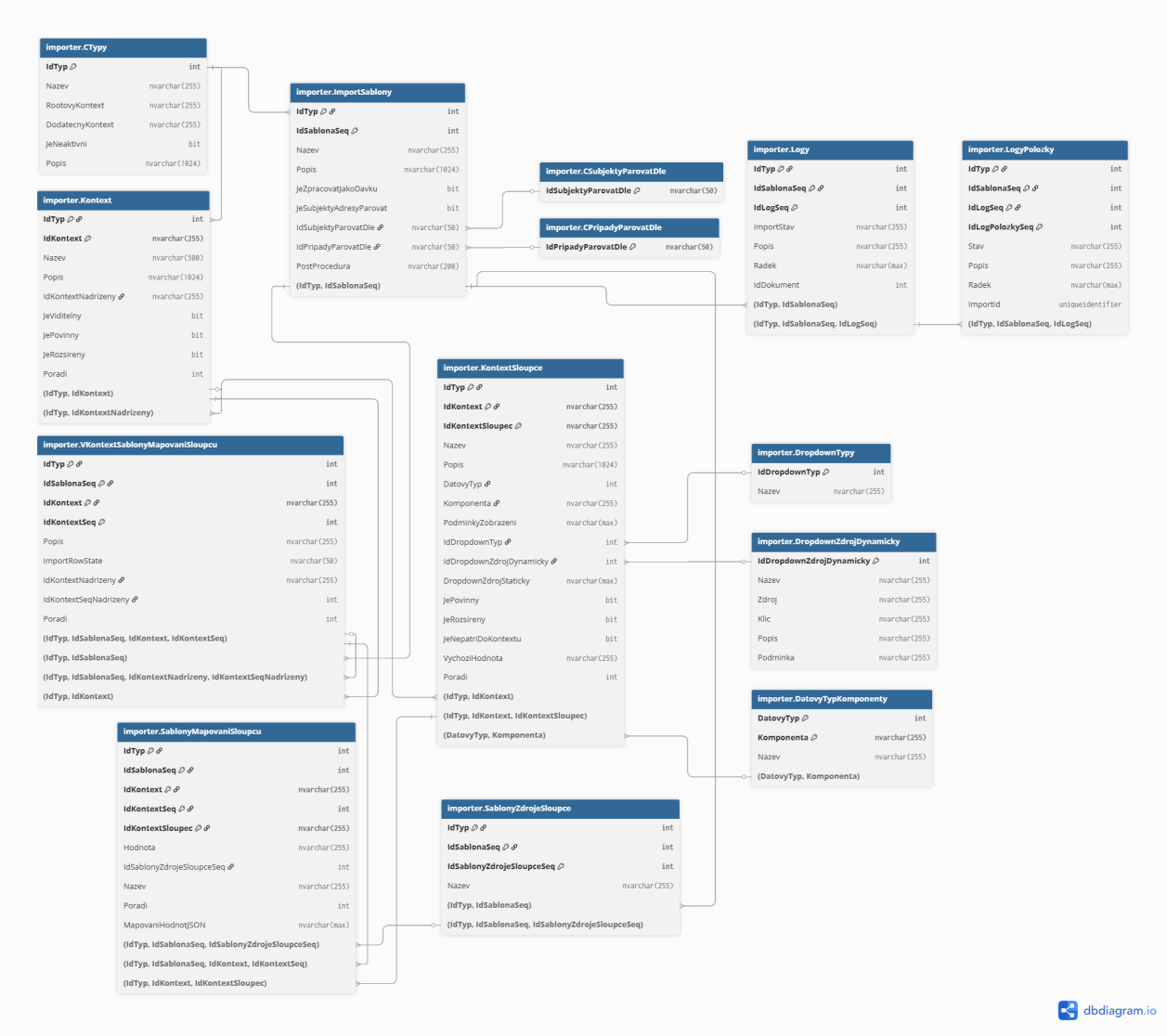
1. **Definice struktury:** Metadata popisující, jaká data lze importovat (kontexty, sloupce).
2. **Konfigurace šablon:** Uložení uživatelských nastavení a mapování.
3. **Provozní data:** Logování průběhu importu a auditní stopy.

3.1.1 Klíčové entity a vztahy

Základním stavebním kamenem je tabulka `CTypy`, která slouží jako kořenový číselník pro logickou separaci dat. Na ni navazuje hierarchická struktura definovaná v tabulce `Kontext`. Významným prvkem návrhu je rekurzivní vazba v tabulce `Kontext` přes sloupec `IdKontextNadrizeny`, což umožňuje modelovat vnořené objekty (např. Subjekt → Adresa → Kontakt).

3.1.2 Systémová infrastruktura a auditování

Všechny tabulky v rámci schématu `importer` sdílejí jednotnou sadu systémových sloupců, které zajišťují auditní stopu a integritu dat. Mezi tyto sloupce patří identifikátory autora záznamu (`SysZadal`), časové značky a sloupec `SysRowVersion` sloužící k optimistickému řízení souběhu.



Obrázek 3.1: Zjednodušený ER diagram databázového schématu Importer

3.1.3 Logické celky datového modelu

Na základě funkčnosti lze datový model rozdělit do několika klíčových oblastí, které společně zajišťují flexibilitu celého importního modulu.

3.1.3.1 Definice hierarchie a struktury

Základem pro definici toho, co lze do systému importovat, jsou tabulky `CTypy`, `Kontext` a `KontextSloupce`.

- **CTypy:** Slouží jako kořenový číselník a diskriminátor pro logickou separaci různých domén importu (např. spisy, subjekty či dokumenty).
- **Kontext:** Reprezentuje kategorii importovatelných dat — například Subjekt, Adresa nebo Kontakt. Díky rekurzivní vazbě `IdKontextNadrizeny` mohou být kontexty vzájemně vnořeny a tvoří tak stromovou strukturu (např. Subjekt → Adresa → Kontakt). Každý kontext zároveň slouží jako nadřazená kategorie pro svá konkrétní datová pole, která jsou evidována v tabulce `KontextSloupce`.
- **KontextSloupce:** Každý záznam v této tabulce představuje jedno konkrétní pole patřící do nadřazeného kontextu — například pole *Jméno* nebo *IČO* v kontextu Subjekt. Kromě datového typu obsahuje také identifikátor UI komponenty, což umožňuje dynamické vykreslování odpovídajících prvků (např. dropdown, datepicker) v prezentační vrstvě aplikace.

3.1.3.2 Konfigurace šablon a mapování

Procesní nastavení a propojení vstupních dat s vnitřní strukturou systému zajišťují entity `ImportSablony` a `SablonyMapovaniSloupcu`. Šablony uchovávají parametry jako režim dávkového zpracování nebo kritéria pro párování existujících subjektů. Tabulka `SablonyMapovaniSloupcu` pak funguje jako komplexní asociační entita, která propojuje tři nezávislé hierarchie: strukturu kontextů, zdrojové sloupce vstupního souboru a konkrétní výskyty kontextů v šabloně. Podporuje definici statických hodnot i pokročilé transformační logiky uložené v JSON formátu. Jejím hlavním úkolem je uložit stav šablony, tedy uchovat všechny její kontexty a sloupce, díky čemuž můžeme do databáze uložit kompletní informaci o hierarchii celé šablony.

3.2 Serverová část a aplikační logika (Add-in)

Serverová část modulu Importér je implementována v jazyce C# jako komponenta systému EFabric. Hlavním úkolem této vrstvy je orchestrace datového toku mezi uživatelským rozhraním, interním API systému Evolio a SQL databází. Logika je rozdělena do dvou funkčních celků (Add-inů), které zajišťují oddělení přípravné a realizační fáze importu.

3.2.1 Procesní flow: Od validace k ostrému zápisu

Zpracování importu probíhá dvoufázově. Tento přístup umožňuje uživateli zkontrolovat připravená data a odhalit případné chyby dříve, než dojde k nevratnému zápisu do produkčních tabulek systému. Porovnání parametrů a chování těchto fází uvádí tabulka 3.1.

Tabulka 3.1: Srovnání fází PrepareImport a DirectImport

Vlastnost	**Fáze PrepareImport**	**Fáze DirectImport**
Účel	Parsování souboru a příprava dat v logu.	Ostrý zápis dat do cílových entit systému.
Hlavní parametr	`PouzeUlozit = true`.	`ImportId` (získané z přípravy).
Zpracování	Hromadné pro celý soubor.	Sekvenční po jednotlivých položkách.
API Endpoint	`/api/import/v5/pripady/`.	`api/import/v5/DirectImport`.

3.2.1.1 Fáze 1: Příprava importu (PrepareImport)

V první fázi odesílá frontend metadata o souboru a zvolenou šablonu mapování. Backend volá standardní importní API systému Evolio s parametrem `PouzeUlozit = true`. Systém v tomto režimu neprovádí zápis do ostrých agend, ale parsuje vstupní Excel/CSV soubor, provádí validace a výsledná data ukládá do dočasných tabulek logu (`Logy` a `LogyPolozky`). Výsledkem tohoto volání je unikátní identifikátor `IdLogSeq` (nebo `ImportId`), který slouží jako klíč pro následné kroky.

3.2.1.2 Fáze 2: Ostrý import (DirectImport)

Jakmile uživatel potvrdí správnost připravených dat, frontend zahájí sekvenční zpracování položek. Pro každou platnou položku je vyvolána metoda `DirectImportAsync`, která využívá dříve získaný identifikátor. Tato fáze volá stejné API rozhraní, ale tentokrát pro finální uložení do databáze.

```
// Metoda zajistuje finalni zapis dat do systemu Evolio
public async Task DirectImportAsync(Guid importId)
{
    // Sestaveni URL adresy pro konkretni ImportId
    var url = string.Format("api/import/v5/DirectImport?ImportId={0}", importId);

    // Odeslani POST pozadavku na API.
    // V teto fazi dojde k fyzickemu vlozeni dat do SQL tabulek systemu.
    await Context.Post(url, string.Empty, CancellationToken.None);
}
```

Listing 3.1: Implementace metody pro provedení ostrého importu

3.2.2 Implementace databázové logiky (SQL Procedura)

Pro zajištění konzistence a výkonu při aktualizaci stavů importu využívá backend uloženou proceduru `ImporterUpdateLogyPolozkySp`. Tato procedura atomicky aktualizuje stav řádku a v případě dokončení celé dávky provádí finalizaci logu.

```sql
CREATE PROCEDURE [importer].[ImporterUpdateLogyPolozkySp]
    @SysRowid [core].[SysRowIdType], -- ID radku v tabulce LogyPolozky
    @Stav nvarchar(255),         -- S (Success) / EC (Error)
    @Popis nvarchar(255) = null  -- Detailni chybova zprava
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @IdTyp INT, @IdSablonaSeq INT, @IdLogSeq INT;

    -- 1. Aktualizace stavu konkretni polozky
    UPDATE importer.LogyPolozky
    SET Stav = @Stav,
        Popis = @Popis
    WHERE SysRowId = @SysRowid;

    -- 2. Kontrola koncovych stavu a aktualizace celkoveho logu
    IF(@Stav = 'S' OR @Stav = 'EC')
    BEGIN
        SELECT @IdTyp = IdTyp, @IdSablonaSeq = IdSablonaSeq, @IdLogSeq = IdLogSeq
        FROM importer.LogyPolozky WHERE SysRowId = @SysRowid;

        -- Overeni, zda jsou jiz vsechny polozky v davce dokoncene
        IF(NOT EXISTS(SELECT 1 FROM importer.LogyPolozky l
                    WHERE l.IdLogSeq = @IdLogSeq AND l.Stav IN ('N','R')))
        BEGIN
            -- Finalizace celeho importniho logu
            UPDATE importer.Logy SET Stav = 'D' WHERE IdLogSeq = @IdLogSeq;
        END
    END
END
```

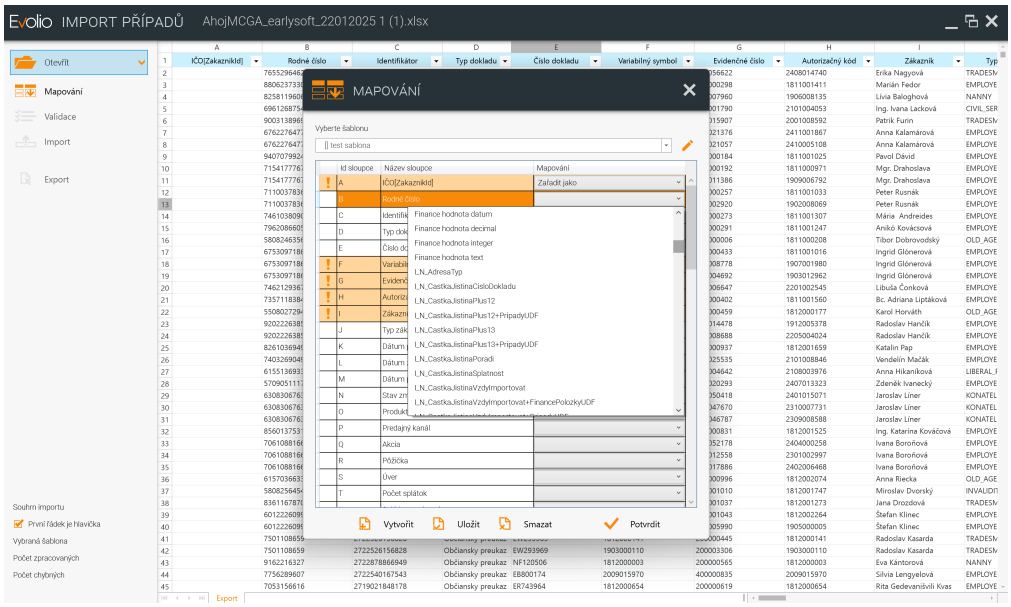
Listing 3.2: Uložená procedura pro aktualizaci stavu položek

Využití této procedury umožňuje backendu efektivně sledovat průběh importu a okamžitě reagovat na výsledky vracené z API. Pokud API vrátí chybu pro konkrétní řádek, backend zavolá proceduru se stavem `'EC'` a popisem chyby, což se ihned promítne do uživatelského rozhraní v reálném čase.

3.3 Uživatelské rozhraní (Frontend)

3.3.1 Motivace a porovnání s původním řešením

Původní modul pro import dat v systému Evolio byl realizován jako desktopová aplikace (WPF). Toto řešení bylo v průběhu let shledáno jako vysoce neintuitivní a pro běžného uživatele bez předchozího proškolení prakticky nepoužitelné (viz Obrázek 3.2). Obsluhu tohoto nástroje zvládali pouze interní firemní konzultanti.

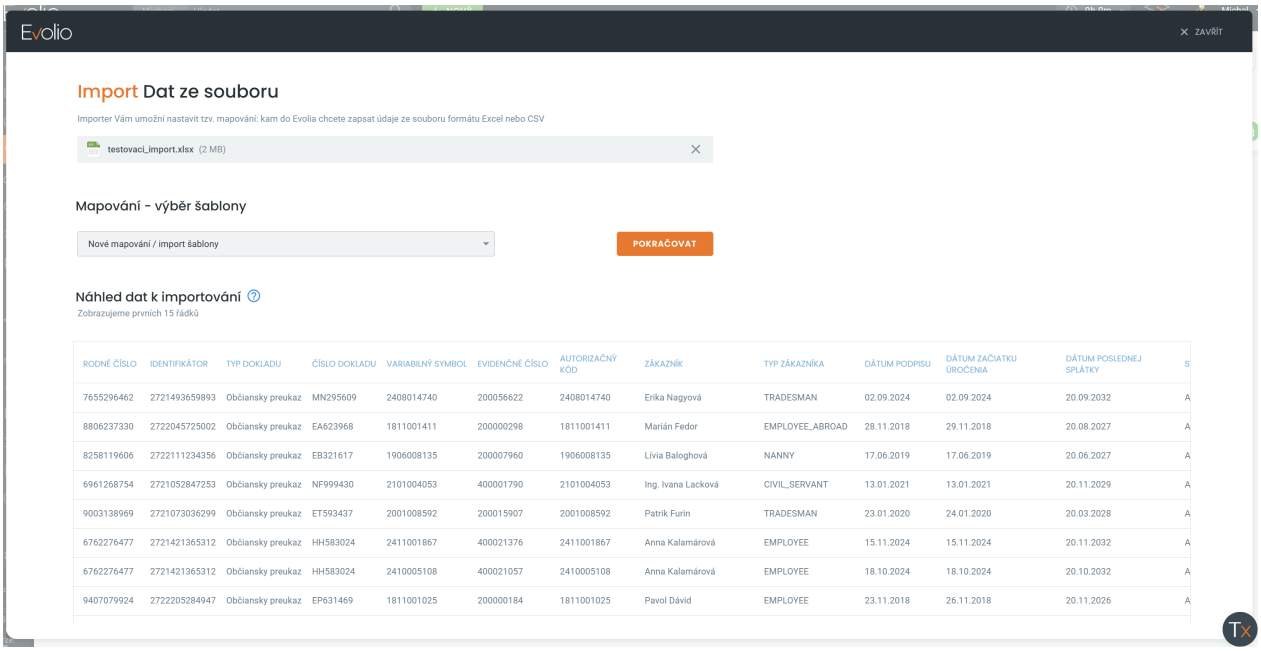


Obrázek 3.2: Ukázka rozhraní původního desktopového modulu (neintuitivní a složité). V rozbaleném dropdownu lze vidět entity, které jsou pro běžného uživatele nicneříkající.

Hlavním cílem vývoje nové webové Single Page Aplikace (SPA) bylo vytvořit moderní a intuitivní nástroj, který umožní klientům provádět importy dat zcela samostatně.

Prezentační vrstva modulu Importér představuje nejrozsáhlejší část realizace. Je navržena jako autonomní feature modul (`ImporterModule`) v rámci enterprise architektury systému Evolio, využívající verzi **Angular 17** [2] a **TypeScript** [3].

3.3.2 Architektura a hierarchie komponent

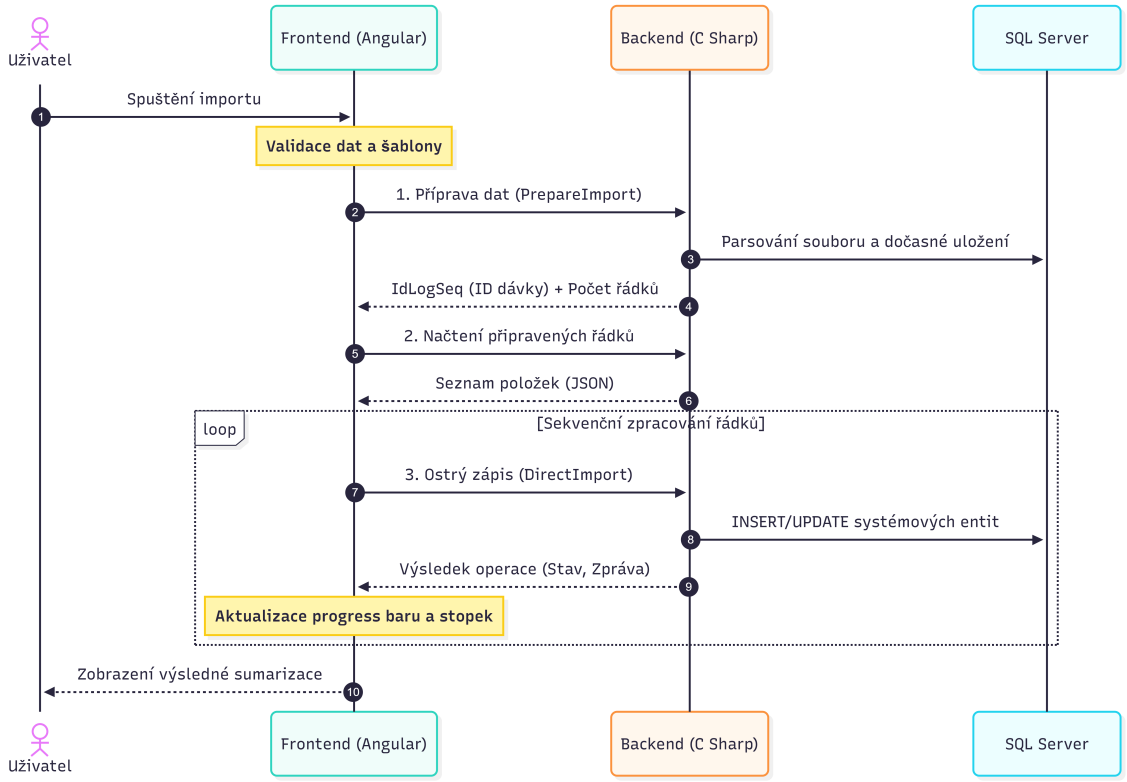


Obrázek 3.3: Dashboard nového modulu Importér se seznamem náhledem dokumentu a výběrem šablony

Modul sestává z osmi specializovaných komponent, které spolupracují na zajištění plynulého uživatelského zážitku. Přehled všech komponent uvádí tabulka 3.2.

Tabulka 3.2: Kompletní přehled komponent modulu Importér

Komponenta	**Účel a hlavní funkce**
`ImporterDetailComponent`	Hlavní dashboard modulu s historií a náhledem.
`ImporterMappingComponent`	Obrazovka pro definici mapování sloupců na kontexty.
`ContextTreeComponent`	Rekurzivní komponenta pro vizualizaci stromu kontextů.
`ImporterDropdownComponent`	Výběrový prvek pro intuitivní přiřazování sloupců.
`ValueMappingModalComponent`	Dialog pro definici transformací hodnot (převod textů na ID).
`DocumentPickerButtonComponent`	Prvek pro nahrávání souborů s podporou validace.
`ImportProgressDialogComponent`	Sleduje průběh importu po řádcích v reálném čase.
`ImportLogDetailComponent`	Detailní prohlížeč výsledků a chyb z proběhlých importů.



Obrázek 3.4: Sekvenční diagram procesu zpracování importu po jednotlivých řádcích

3.3.3 Moderní reaktivní vzory: Signály a RxJS

Aplikace využívá kombinaci tradiční knihovny RxJS [6] a nového reaktivního primitiva **Signals**, představeného v Angularu 17. Signály jsou použity pro správu synchronního stavu, který nevyžaduje komplexní transformace, což zjednodušuje kód a zvyšuje čitelnost.

V rámci `ImporterService` jsou signály využity pro sledování aktuálního kontextu:

```
// Definice signálu pro sledování vybrané šablony
public selectedSablonaId = signal<number | null>(null);

// Výpočetní signál závislý na změně ID
public isSablonaSelected = computed(() => this.selectedSablonaId() !== null);
```

Listing 3.3: Využití signálů pro správu stavu modulu

Pro asynchronní operace, jako je paralelní načítání číselníků při inicializaci, je nadále využíváno RxJS. Operátor `forkJoin` zajišťuje, že se aplikace vykreslí až v momentě, kdy jsou k dispozici všechna potřebná metadata:

```
forkJoin({
  typy: this.emap.get('cTypy', { IdTyp: this.idTyp() }),
  kontexty: this.emap.get('kontext', { IdTyp: this.idTyp() }),
  sablony: this.emap.get('importSablony', { IdTyp: this.idTyp() })
}).pipe(takeUntil(this.destroy$)).subscribe(data => {
  this.state.update(s => ({ ...s, ...data }));
});
```

Listing 3.4: Paralelní inicializace dat pomocí forkJoin

3.3.4 Optimalizace výkonu pomocí OnPush strategie

Z důvodu vysokých nároků na výkon při zobrazování rozsáhlých mapovacích struktur (stromy s desítkami uzlů) byla u všech komponent s vyššími nároky na výkon nasazena strategie **ChangeDetectionStrategy.OnPush**.

Tato optimalizace instruuje Angular, aby kontroloval změny v komponentě pouze v případě, že se změní její vstupní reference (`@Input`) nebo je změna explicitně vyžádána. Tím se výrazně redukuje počet cyklů detekce změn při interakci uživatele s formulářem.

```
@Component({
  selector: 'ev-importer-mapping',
  templateUrl: './importer-mapping.component.html',
  styleUrls: ['./importer-mapping.component.scss'],
```

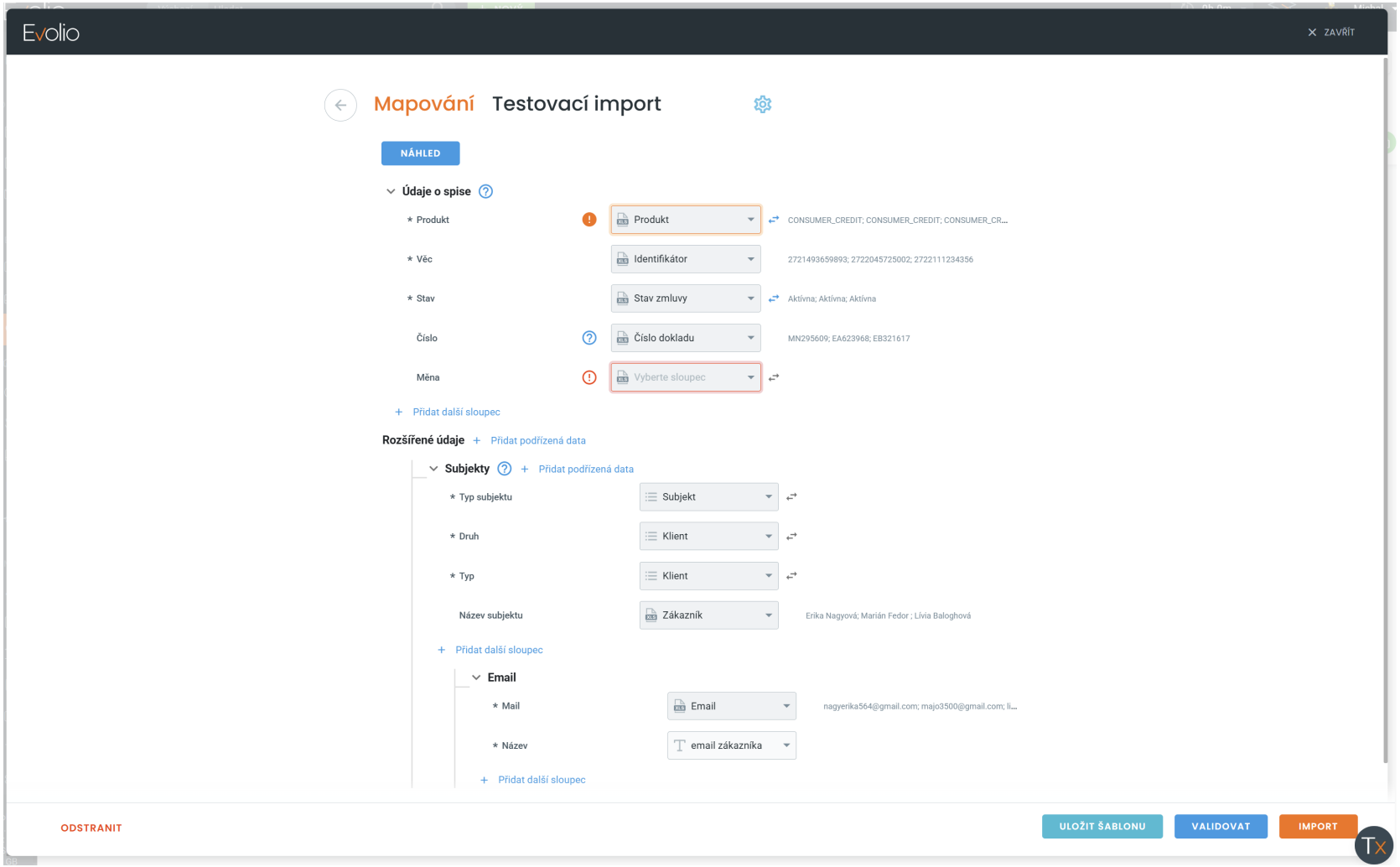
```
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class ImporterMappingComponent { ... }
```

Listing 3.5: Definice komponenty s OnPush strategií

3.3.5 Implementace komplexní mapovací logiky

Jádrem modulu je `ImporterMappingComponent` (viz Obrázek 3.5), která umožňuje uživateli interaktivně sestavit strukturu importu. Na rozdíl od původního desktopového řešení se zde uplatňuje princip *on-demand* budování struktury:

- **Dynamické instance:** Uživatel může přidávat více instancí stejného kontextu (např. více adres k jednomu subjektu). Každá instance má vlastní stav mapování sloupců.
- **Interaktivní správa:** Stav sbalení a rozbalení jednotlivých sekcí je udržován v `Set<string>`, což zajišťuje rychlou manipulaci i při velkém počtu prvků.
- **Datová transformace:** Pomocí *pure pipes* dochází k transformaci dat bez nutnosti přepočítávání při každém cyklu detekce změn.

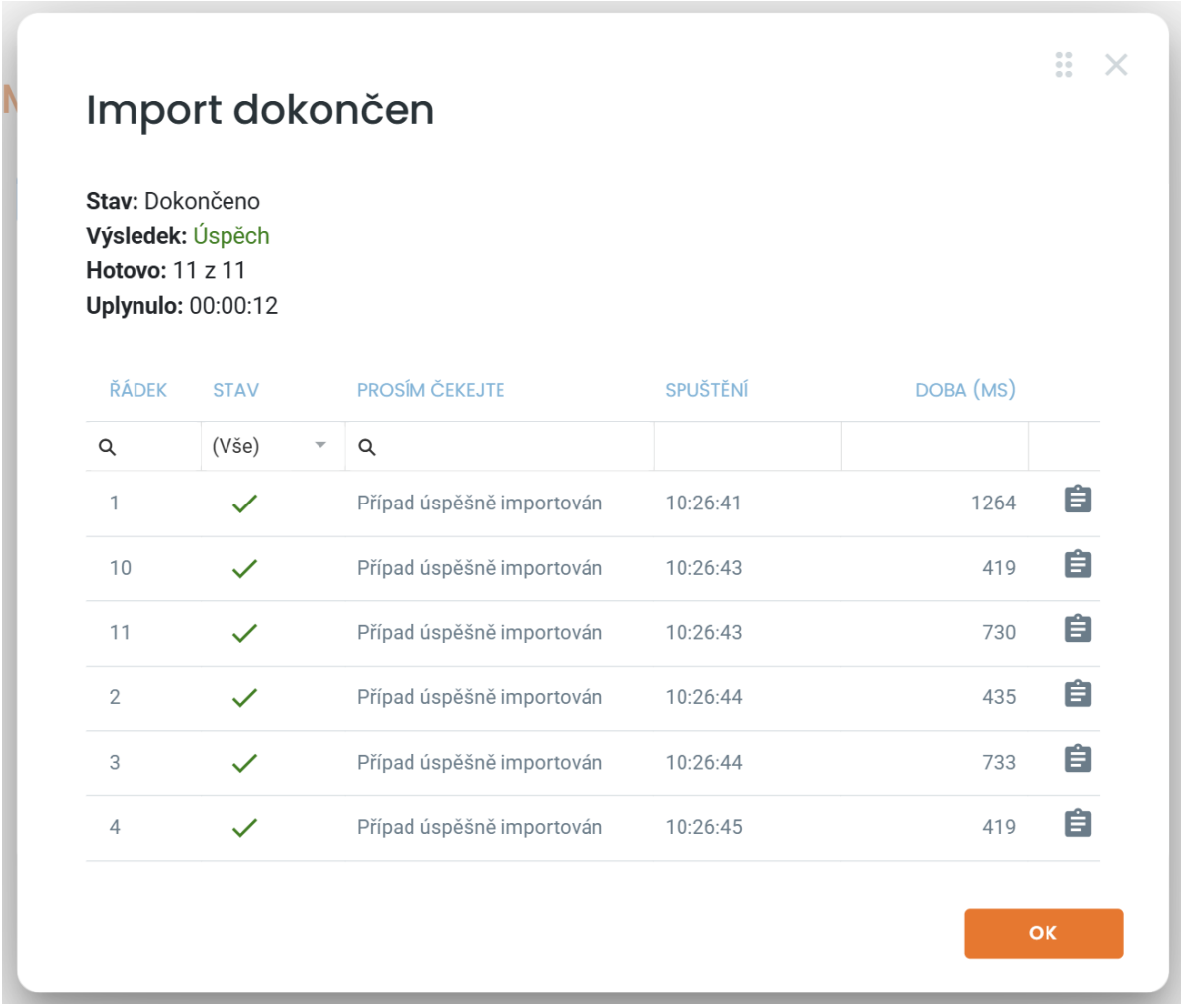


Obrázek 3.5: Nové rozhraní pro interaktivní konfiguraci mapování sloupců

3.3.6 Real-time monitoring a UX prvky

Proces importu probíhá sekvenčně, přičemž uživatel je o postupu informován prostřednictvím `ImportProgressDial`
Tato komponenta kombinuje několik technologií pro zajištění plynulé zpětné vazby:

- **Sledování času:** Využití knihovny *moment.js* v sekundovém intervalu pro zobrazení doby trvání importu.
- **Inkrementální DataGrid:** Výsledky jsou do tabulky (*DevExtreme DataGrid*) přidávány okamžitě po dokončení požadavku na konkrétní řádek.
- **Vizuální indikátory:** Namísto statických spinnerů jsou v dashboardu využity **skeleton loadingy**, které predikují strukturu obsahu a snižují kognitivní zátěž uživatele při čekání na data.



Obrázek 3.6: Ukázka úspěšného importu

Kapitola 4

Vývoj AI rozhraní a řešení FE issues

Vedle hlavního projektu Importér byla významná část času věnována práci na klientských požadavcích a údržbě existujících systémů. Tato kapitola popisuje vývoj specializovaného AI modulu a řešení specifických chyb v uživatelském rozhraní.

4.1 Implementace AI asistenta v prostředí AngularJS

Jedním z nejnáročnějších úkolů byl vývoj inteligentního chatu pro specifického klienta, jehož informační systém je postaven na starší technologii **AngularJS (v1.x)** [7]. Cílem bylo integrovat moderní jazykové modely (LLM) do tohoto prostředí a umožnit uživatelům nejen konverzaci, ale i analýzu nahraných dokumentů.

Je důležité podotknout, že framework AngularJS je v dnešní době již považován za technologii za hranicí své životnosti (End-of-Life) a v rámci standardních firemních procesů jej pro nové projekty již nevyužíváme. Volba této technologie byla v tomto případě čistě pragmatická a vynucená architekturou stávajícího informačního systému klienta. Cílem bylo vyhovět požadavku na integraci moderních funkcí do již zavedené aplikace a zajistit její další rozvoj bez nutnosti okamžité a nákladné migrace celého systému na modernější platformu.

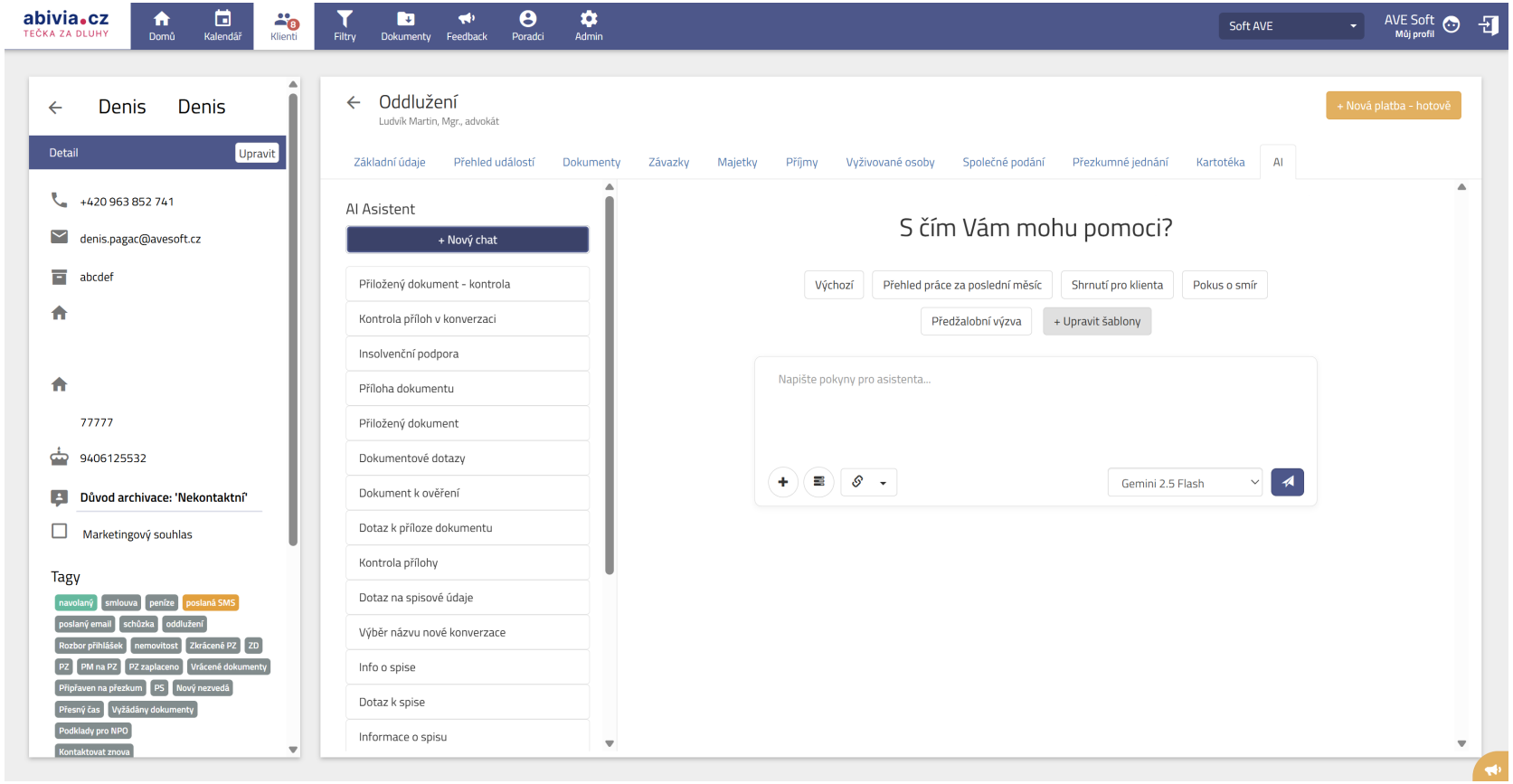
4.1.1 Architektura a komunikace v reálném čase

Vzhledem k požadavku na plynulé zobrazování odpovědi (podobně jako v ChatGPT) nebylo možné čekat na vygenerování celé odpovědi na serveru. Řešení využívá technologii **SignalR** [8] a Azure Notification Hub pro streamování jednotlivých tokenů textu v reálném čase.

Datový tok je následující:

1. Frontend odešle HTTP POST požadavek s dotazem a ID vybraného modelu.

2. Backend zahájí komunikaci s AI API (např. OpenAI/Azure).



Obrázek 4.1: Výchozí rozhraní AI asistenta s nabídkou předdefinovaných šablon

3. Jakmile AI generuje text, backend posílá jednotlivé fragmenty (chunks) přes SignalR Hub zpět na klienta.

4. Frontend naslouchá na eventu `openAIStream` a postupně skládá odpověď.

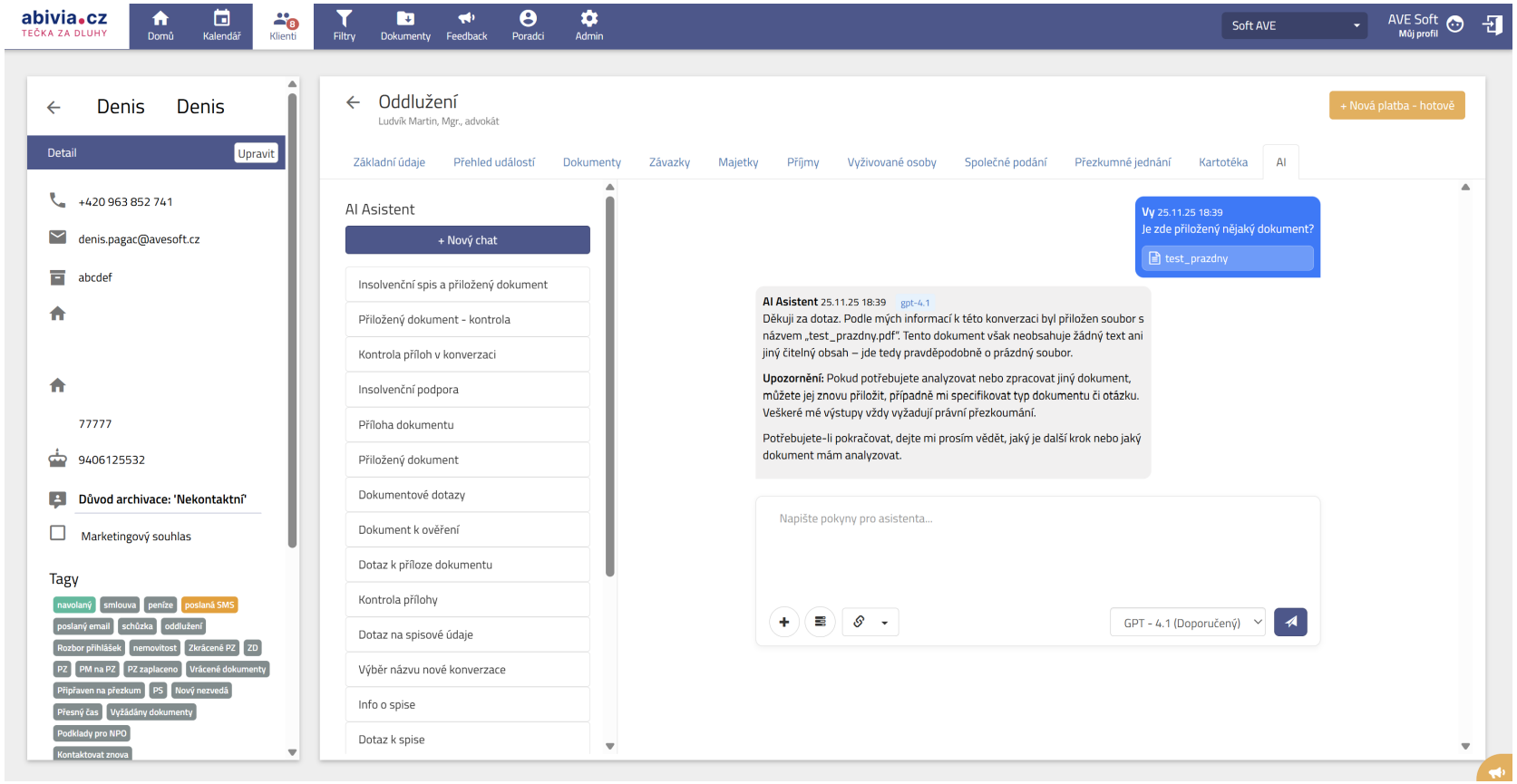
```
// Naslouchání na SignalR notifikace
hub.on('openAIStream', function(message) {
    $rootScope.$broadcast('openAIStream', message);
});

// Skládání zprávy v kontroleru
$scope.$on('openAIStream', function(event, chunk) {
    // Přidání nového tokenu k existujícímu textu
    $scope.streamingMessage.text += chunk;

    // Vynucení překreslení UI (AngularJS digest cycle)
    if (!$scope.$$phase) $scope.$apply();
});
```

Listing 4.1: Zpracování streamované odpovědi v AngularJS

4.1.2 Práce s dokumenty a formátování

Klíčovým požadavkem byla analýza právních dokumentů. Před odesláním dotazu probíhá asynchronní upload souborů, které jsou na backendu převedeny do formátu vhodného pro LLM kontext.

Pro správné zobrazení strukturovaného textu (odrážky, tučné písmo, kód) byla využita knihovna **marked.js**, která v reálném čase převádí Markdown syntaxi na HTML.



Obrázek 4.2: Ukázka probíhající konverzace s formátováním textu a analýzou dokumentu

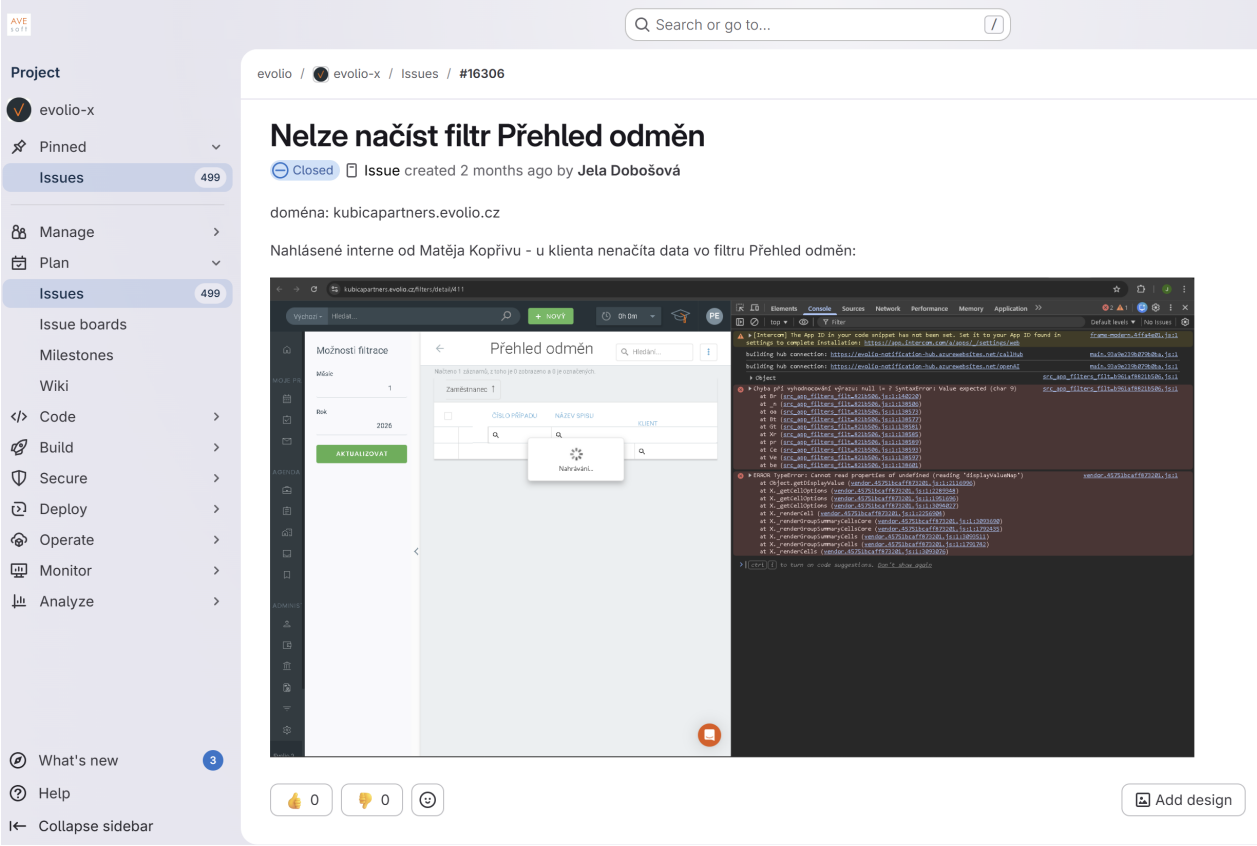
4.2 Řešení chyb v uživatelském rozhraní (Bugfixing)

Nedílnou součástí praxe byla analýza a opravy chyb v systému Evolio. Příkladem komplexního problému byla chyba v komponentě Přehledy (založené na knihovně DevExpress [9]), kde docházelo k selhání načítání filtrů.

4.2.1 Analýza problému: Konflikt seskupování a ukotvení

Uživatelé hlásili, že u určitých přehledů se nenačetli data v tabulce a došlo k nekonečnému načítání. Debugováním bylo zjištěno, že problém nastává v momentě, kdy je sloupec nastaven jako **ukotvený** (Fixed) a zároveň se podle něj **seskupuje** (Grouped).

Komponenta DevExpress v této specifické konfiguraci nedokázala správně vypočítat pozici elementů, což vedlo k chybě v konzoli a nenačtení dat.



Obrázek 4.3: Ukázka zadaného issue na gitlabu

4.2.2 Implementace opravy

Oprava spočívala v úpravě logiky, která generuje konfiguraci pro sloupce tabulky. Bylo nutné zajistit, aby se vlastnost `fixed` automaticky deaktivovala, pokud je sloupec použit pro seskupování.

```
// Původní problematický kód
// fixedPosition: colcf.fixed !== null ? colcf.fixed.toLowerCase() : null,
// fixed: colcf.fixed !== null,

// Nová logika s ošetřením konfliktu
const isFixed = colcf.fixed !== null &&
               colcf.fixed.toLowerCase() !== 'none' &&
               !colcf.grouped; // Klicova podminka: nesmi byt seskupeno

// Aplikace do konfigurace
const columnConfig = {
    // ... ostatní vlastnosti
    fixedPosition: isFixed ? colcf.fixed.toLowerCase() : null,
    fixed: isFixed
};
```

Listing 4.2: Oprava konfliktu vlastností Fixed a Grouped

Tato úprava zajistila stabilitu komponenty i v případě, že uživatel (nebo uložený přehled) definoval tuto konfliktní kombinaci vlastností.

Kapitola 5

Automatizované testování

Pro zajištění kvality a stability systému Evolio existuje v rámci firemní infrastruktury sada automatizovaných UI testů. Tyto testy simulují reálné chování uživatele v prohlížeči a slouží jako regresní testování po nasazení nové verze aplikace. V rámci praxe jsem se do tohoto projektu zapojil — rozšiřoval jsem stávající sadu o nové testovací scénáře a průběžně pečoval o funkčnost již existujících testů.

5.1 Strategie a nástroje

Jako technologický základ byl zvolen framework **Selenium WebDriver** [10], který je v .NET prostředí standardem pro automatizaci webových prohlížečů. Pro samotné spouštění testů a validaci podmínek (assertions) je využit framework **NUnit** [11] (atributy `[Test]`, `[SetUp]`).

5.1.1 Architektura Screenplay Pattern

Na rozdíl od tradičního přístupu Page Object Model (POM) využívá tento projekt modernější návrhový vzor Screenplay Pattern, implementovaný pomocí knihovny **Boa.Constrictor** [12]. Tento přístup se zaměřuje na to, co uživatel (Actor) dělá, spíše než na strukturu stránek.

Základní stavební kameny architektury jsou:

- **Actor (Tester):** Entita provádějící test. V našem případě je instanciována v `BaseTest` třídě.
- **Task:** Složitější úloha skládající se z více kroků (např. `Login.WithSessionCredentials()`).
- **Interaction:** Atomická akce (např. `Click.On(...)`, `SendKeys.To(...)`).
- **Question:** Dotaz na stav aplikace (např. `Appearance.Of(...)`, `ValueAttribute.Of(...)`).

5.2 Implementace testovacích scénářů

Testy jsou organizovány do logických celků podle agend (Podatelna, Případy, Subjekty). Následující ukázka demonstruje strukturu CRUD testu pro vytvoření nového subjektu.

5.2.1 Příklad: Vytvoření subjektu

Testovací metoda dodržuje princip **Arrange-Act-Assert**. Využívá fluent syntaxi, která činí test čitelným téměř jako přirozený jazyk.

```
[Test]
public void CreateSubject_Manually_ShouldSucceed()
{
    // Arrange: Příprava dat
    string name = "Testovací Subjekt " + Guid.NewGuid();
    string ico = "12345678";

    // Act: Provedení akcí (Navigace -> Vyplnění formuláře -> Uložení)
    Tester.AttemptsTo(Navigate.ToUrl(SubjectPage.Url));
    Tester.AttemptsTo(Click.On(SubjectPage.AddButton));

    Tester.AttemptsTo(SendKeys.To(SubjectForm.NameInput, name));
    Tester.AttemptsTo(SendKeys.To(SubjectForm.IcoInput, ico));

    Tester.AttemptsTo(Click.On(SubjectForm.SaveButton));

    // Assert: Ověření výsledku
    // Čekáme, až se objeví notifikace o úspěchu
    Tester.WaitsUntil(Appearance.Of(SubjectPage.SuccessToast), IsEqualTo.True());

    // Ověření, že se subjekt načetl v detailu
    Tester.AskingFor(Text.Of(SubjectPage.DetailHeader)).Should().Contain(name);
}
```

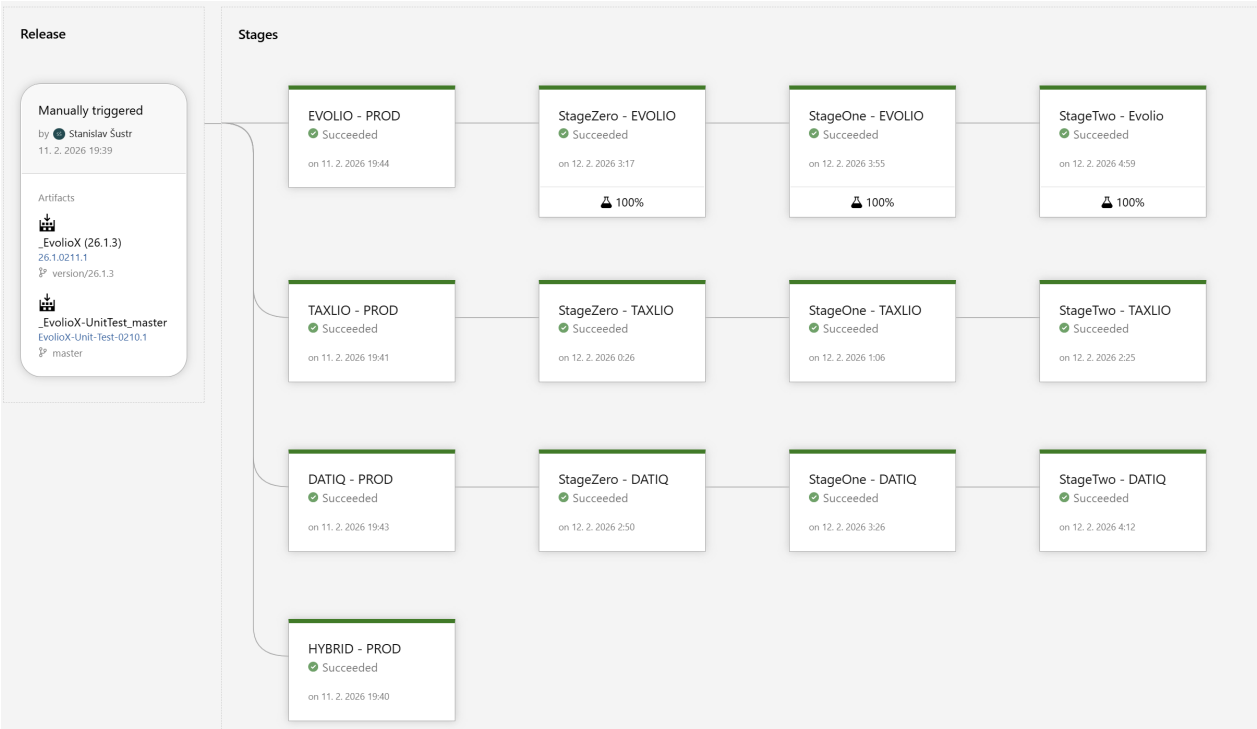
Listing 5.1: Ukázka testu vytvoření subjektu pomocí Screenplay patternu

5.3 Integrace do CI/CD (Azure DevOps)

Testovací projekt je integrován do procesu kontinuální integrace a nasazování (CI/CD) na platformě **Azure DevOps** [13], kde jsem se podílel na správě buildovacích a release pipeline.

5.3.1 Spouštění testů

Testy jsou automaticky spouštěny po každém nasazení nové verze do testovacího prostředí. Běží na vzdáleném virtuálním stroji v prohlížeči Google Chrome.



Obrázek 5.1: Ukázka úspěšného běhu automatizovaných testů v release pipeline Azure DevOps

5.3.2 Diagnostika chyb

Pro efektivní řešení problémů je implementován mechanismus, který v případě pádu testu:

1. Automaticky pořídí **screenshot** obrazovky v momentě chyby.
2. Uloží systémové logy a stack trace.
3. Tyto artefakty připojí k výsledku testu v Azure DevOps Dashboardu.

Tato automatizace výrazně zrychluje proces debugování, protože umožňuje okamžitě identifikovat, zda chyba nastala v aplikaci (např. validační hláška) nebo v testu samotném.

Kapitola 6

Vazba praxe na studium

Odborná praxe mi umožnila konfrontovat teoretické znalosti získané studiem na Fakultě elektrotechniky a informatiky VŠB-TUO s reálnými požadavky softwarového vývoje. V mnoha oblastech jsem mohl přímo navázat na absolvované předměty, zatímco jiné domény vyžadovaly intenzivní samostudium. Tato kapitola se zaměřuje na konkrétní využití školní látky v praxi; přehled nově osvojených technologií a chybějících znalostí je věnována samostatná kapitola 7.

6.1 Uplatnění teoretických znalostí

Nejvýznamnějším přínosem studia pro tuto praxi byla znalost objektově orientovaného programování a databázových systémů. Konkrétně jsem využil znalosti z následujících předmětů:

- **Programování v C# 1 a 2:** Znalost syntaxe jazyka C#, typového systému a asynchronního programování byla nezbytná pro vývoj backendové části (Add-in) a psaní automatizovaných testů. Konkrétním příkladem bylo využití `async`/`await` a generických typů při implementaci metody `DirectImportAsync`, kde bylo nutné správně zpracovat asynchronní volání interního API systému. Bez pevného základu z těchto předmětů bych nebyl schopen rychle pochopit strukturu existujícího produkčního kódu a do něj přispívat.

- **Databázové systémy 1 a 2:** Principy návrhu databází jsem využil při spolupráci na návrhu schématu pro modul Importér. Pomohlo mi, že jsem rozuměl tomu, proč je důležité správně definovat vztahy mezi tabulkami a zajistit integritu dat již na úrovni databáze. Stejně tak mi znalost SQL usnadnila psaní uložených procedur a orientaci v existujících dotazech systému Evolio.

- **Funkcionální programování (FPR):** Ačkoliv je většina projektu psána v imperativních jazycích, principy rekurze probírané v tomto předmětu byly klíčové při implementaci hierarchického stromu kontextů (*Context Tree*). Zobrazení vnořených struktur (např. Subjekt →

Adresa → Kontakt) vyžadovalo rekurzivní průchod daty jak v databázovém schématu (rekurzivní vazba `IdKontextNadrizeny`), tak i ve frontendové komponentě `ContextTreeComponent`, která se sama volá pro každou úroveň stromu. Tento koncept jsem si pevně osvojil právě díky výuce FPR.

- **Správa softwarových projektů (SSP):** Základy verzování kódu pomocí systému Git, které byly představeny v tomto předmětu, jsem využíval denně při práci s větvemi (branches) a správě zdrojových kódů. Ve firemním prostředí jsem však narazil na výrazně přísnější kulturu kolem větvení — každý úkol má vlastní větev, merge requesty procházejí code review minimálně jednoho kolegy a commity musí dodržovat konvence. Tato praxe nebyla ve výuce zdůrazněna, ale ocenění jejího smyslu přišlo velmi rychle.

6.2 Přechod od teorie k praxi

Přestože výše zmíněné předměty poskytly solidní základ, praxe odhalila, že rozdíl mezi akademickým a průmyslovým pojetím programování není jen v technologiích, ale především v měřítku a kontextu. Školní projekty jsou zpravidla budovány od nuly, mají jasně ohraničený rozsah a nejsou svázány existující architekturou. V praxi naopak probíhá vývoj v rámci rozsáhlého systému s mnohaletou historií, kde každá změna musí respektovat zavedené konvence, nezlomit existující funkčnost a projít revizí kolegů.

Tato zkušenost pro mě byla klíčová: potvrdila, že teoretický základ ze školy je nezbytnou podmínkou, nikoli však postačující. Schopnost číst a pochopit cizí produkční kód, orientovat se v architektuře systému, který jsi nevytvořil, a komunikovat technická rozhodnutí v týmu — to jsou dovednosti, které se nedají plně natrénovat v akademickém prostředí a které jsem si mohl osvojit právě až v průběhu praxe.

Kapitola 7

Rozvoj kompetencí a chybějící znalosti

Přechod do firemního prostředí odhalil rozdíly mezi akademickými projekty a vývojem komerčního softwaru. Tato kapitola shrnuje technologie, které jsem se musel naučit samostudiem v průběhu praxe, a procesní aspekty, které ve výuce chyběly nebo byly pokryty jen povrchně.

7.1 Nově osvojené technologie

Největší výzvou byl vývoj moderního frontendu. Ačkoliv předmět *Tvorba aplikací pro mobilní zařízení I* poskytuje základy jazyka JavaScript (Vanilla JS), pro práci na projektu Importér bylo nutné ovládnout komplexní ekosystém frameworku Angular a jazyka TypeScript.

7.1.1 Reaktivní programování s RxJS

Největším technologickým skokem bylo pochopení reaktivního programování. Koncept `Observable` — datového toku, na který se přihlásíme k odběru a reagujeme na příchozí hodnoty — byl zpočátku velmi odlišný od imperativního stylu, na který jsem byl zvyklý ze školních projektů.

V praxi jsem se nejčastěji setkával s operátory jako `forkJoin` pro paralelní volání více API najednou, `switchMap` pro zrušení předchozího požadavku při rychlém přepisování vstupu a `takeUntil` pro bezpečné odhlášení od odběru při zničení komponenty. Bez jejich pochopení by kód byl buď nefunkční, nebo by docházelo k tzv. memory leakům.

7.1.2 Angular architektura

Angular jako framework přináší přísnou strukturu, která zpočátku působí jako zbytečná komplexita, ale při vývoji rozsáhlejšího modulu se ukázala jako velká výhoda. Musel jsem pochopit systém Dependency Injection, který řídí, jak si komponenty a služby předávají závislosti, životní cyklus komponent (hooks jako `ngOnInit`, `ngOnDestroy`) a způsob, jakým Angular detekuje změny v datech a kdy překresluje UI.

Zvláštní pozornost si vyžádala strategie **OnPush**, kterou jsem musel nasadit u všech náročnějších komponent modulu Importér kvůli výkonu. Pochopení toho, kdy Angular komponentu překreslí a kdy ne, bylo klíčové pro odladění chyb, kde se UI neaktualizovalo po změně dat.

7.1.3 Azure Service Fabric a architektura mikroslužeb

Systém Evolio je postaven na platformě Azure Service Fabric, kde každá funkcionalita existuje jako samostatná služba (Actor nebo Service) s vlastním životním cyklem. Pro mě jako studenta zvyklého na monolitické aplikace bylo zpočátku obtížné pochopit, jak spolu tyto služby komunikují, jak se ladí chyba, která přechází přes hranice více služeb, a jak správně nastavit lokální vývojové prostředí tak, aby všechny závislosti fungovaly.

7.2 Chybějící procesní znalosti

Vedle technologií jsem si musel osvojit i řadu procesních dovedností, které jsou ve firemním prostředí samozřejmostí, ale ve školní výuce nejsou systematicky pokryty.

7.2.1 Code Review a kultura zpětné vazby

Každý kód, který jsem napsal, procházel před začleněním do hlavní větve revizí kolegy. Zpočátku bylo náročné přijímat připomínky k vlastnímu kódu a chápat je jako přirozenou součást procesu, nikoliv jako kritiku. Postupně jsem si ale zvykl na to, že code review odhaluje věci, které autor sám přehlédne, a naučil jsem se z komentářů kolegů aktivně čerpat znalosti o firemních konvencích a architektonických rozhodnutích.

7.2.2 Práce v rozsáhlé sdílené kódové základně

Na rozdíl od školních projektů, kde celý kód píše jeden člověk nebo malá skupina, systém Evolio tvoří stovky souborů vyvíjených souběžně více vývojáři. Naučil jsem se orientovat v cizím kódu, dohledávat kontext změn přes historii Gitu a dbát na to, aby moje změny nezasáhly do oblastí, na kterých paralelně pracuje kolega — čemuž slouží právě striktní větvení a pravidelné mergování.

Kapitola 8

Zhodnocení výsledků a přínos praxe

Cílem praxe bylo nejen zapojení do běžného chodu firmy, ale především dodání konkrétních softwarových řešení. Níže uvádím aktuální stav jednotlivých projektů.

8.1 Modul Importér

V době psaní této zprávy se nová webová verze Importéru nachází ve fázi **intenzivního testování**. Aplikace je plně funkční a probíhají konzultace s Team Leaderem a Product Ownerem ohledně doladění vizuálních detailů a user experience (UX).

Do testování jsou zapojeni také firemní konzultanti, kteří budou systém v budoucnu nejvíce využívat. Jejich zpětná vazba slouží k identifikaci posledních nedostatků před nasazením do ostrého provozu pro všechny klienty.

8.2 AI Rozhraní

Modul AI chatu pro specifického klienta byl úspěšně dokončen, nasazen do produkčního prostředí a je klientem aktivně využíván. Řešení splnilo požadavky na analýzu dokumentů a streamovanou komunikaci, přičemž klient vyjádřil s výsledkem spokojenost.

8.3 Automatizované testy

Sada automatizovaných testů byla integrována do CI/CD pipeliny a spouští se s každým vydáním nové verze. Testy již prokázaly svou reálnou hodnotu při odhalování regresních chyb.

Konkrétním příkladem úspěšného záchytu chyby byla situace, kdy testy odhalily nefunkčnost modálního okna pro editaci případu, pokud neobsahoval žádné subjekty. Chyba způsobovala "nekonečné načítání"a zablokování formuláře. Díky automatizaci byla tato kritická chyba zachycena ještě před nasazením k zákazníkům.

Kapitola 9

Závěr

Cílem mé odborné praxe ve společnosti AVE Soft s.r.o. bylo zapojení se do reálného vývojového procesu a realizace konkrétních softwarových úkolů v rámci systému Evolio. Hlavním výstupem byl návrh a implementace webového rozhraní pro modul Importér, který nahrazuje zastaralé desktopové řešení.

Během praxe se podařilo vytvořit moderní Single Page Aplikaci v technologii Angular, která komunikuje s nově vytvořeným backendovým doplňkem. Ačkoliv je modul momentálně ve fázi finalizace a testování, již nyní nabízí intuitivnější uživatelské rozhraní a zjednodušuje proces importu dat. Vedlejší úkoly, jako implementace AI chatu a tvorba automatizovaných testů, byly úspěšně dokončeny a nasazeny do provozu, kde přinášejí reálnou hodnotu jak firmě, tak jejím klientům.

Tato zkušenost pro mě byla klíčová pro pochopení fungování softwarové firmy. Osvojil jsem si moderní technologie, které nebyly součástí výuky (Angular, RxJS), a naučil se pracovat v týmu s využitím pokročilých nástrojů pro správu verzí a CI/CD. Praxe mi potvrdila, že kvalitní teoretický základ ze školy je nezbytný, ale pro uplatnění v oboru je nutné jej neustále rozvíjet samostudiem a reálnou prací na projektech.

Literatura

1. MICROSOFT. *.NET Documentation* [online]. [cit. 2026-03-01]. Dostupné z: https://learn.microsoft.com/en-us/dotnet/.
2. GOOGLE LLC. *Angular - The web development framework for building the future* [online]. [cit. 2026-03-01]. Dostupné z: https://angular.io/docs.
3. MICROSOFT. *TypeScript: JavaScript With Syntax For Types* [online]. [cit. 2026-03-01]. Dostupné z: https://www.typescriptlang.org/.
4. MICROSOFT. *C# Documentation* [online]. [cit. 2026-03-01]. Dostupné z: https://learn.microsoft.com/en-us/dotnet/csharp/.
5. MICROSOFT. *SQL Server Technical Documentation* [online]. [cit. 2026-03-01]. Dostupné z: https://learn.microsoft.com/en-us/sql/sql-server/.
6. RXJS CORE TEAM. *RxJS: Reactive Extensions Library for JavaScript* [online]. [cit. 2026-03-01]. Dostupné z: https://rxjs.dev/.
7. GOOGLE LLC. *AngularJS - Superheroic JavaScript MVW Framework* [online]. [cit. 2026-03-01]. Dostupné z: https://angularjs.org/.
8. MICROSOFT. *Introduction to SignalR* [online]. [cit. 2026-03-01]. Dostupné z: https://learn.microsoft.com/en-us/aspnet/core/signalr/introduction.
9. DEVELOPER EXPRESS INC. *DevExpress UI Components* [online]. [cit. 2026-03-01]. Dostupné z: https://www.devexpress.com/.
10. SOFTWARE FREEDOM CONSERVANCY. *Selenium WebDriver* [online]. [cit. 2026-03-01]. Dostupné z: https://www.selenium.dev/documentation/webdriver/.
11. NUNIT.ORG. *NUnit Documentation* [online]. [cit. 2026-03-01]. Dostupné z: https://docs.nunit.org/.
12. PALO ALTO NETWORKS. *Boa.Constrictor: The .NET Screenplay Pattern* [online]. [cit. 2026-03-01]. Dostupné z: https://github.com/PaloAltoNetworks/boa-constrictor.
13. MICROSOFT. *Azure DevOps Documentation* [online]. [cit. 2026-03-01]. Dostupné z: https://learn.microsoft.com/en-us/azure/devops/.